15,471,856 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

Template Messaging Framework



Asif
5 Aug 2015 CPOL

Rate me: 0.00/5 (No votes)

Framework helps resolve Template Codes present in any text at Runtime

Download MessageTemplateEngine.rar - 54.4 KB

Download MessageTemplateEngine.zip - 58.3 KB

Content

- Introduction
- The Problem
- Solution
  - First Iteration
  - Second Iteration
  - Third Iteration – Build a Messaging Library
- Points Of Interest
  - Configuration Source in the Table
- History

Introduction

In the development world every now and then we face business requirements of manipulating messages whether in the form of SMS, Emails, Word Documents, PDFs, reports etc. Most of the time designed solutions are not well thought, not maintainable and hard coded. What average Developers

usually do is hard code the text in the code to accomplish the business requirements. Since no body review the code hence this will become part of the production code and stay alive until a code review iteration identify it.

A search for a standard solution suggests adopting Template Codes style messages which promises optimum solution. Template code indeed can play vital role because of their highlight maintainable nature and complete isolation from the code. They have enormous power to act as a basis for building large production scale messaging application. This article initially explains developers way of addressing the problem and then apply optimization iteration over the same code improving it to the end that it leads to full production scale library.

The Problem

Most of the developer's usually end up coding the requirement using below straight forward approach.

C# Shrink ▲

```
static void Attemp1()
{
    XmlDocument xmlDocument = new XmlDocument();
    StringBuilder sb = new StringBuilder();

    string xmlData = "<d>" +
                        "<Customers>" +
                           "<Customer Name='Irfan'/>" +
                               "<Orders Total='99999.99'>" +
                                   "<Order No='1' Type='ABC' Amount='1111'/>" +
                                   "<Order No='2' Type='DEC' Amount='222'/>" +
                               "</Orders>" +
                        "</Customers>" +
                     "</d>";
    xmlDocument.LoadXml(xmlData);

    sb.AppendLine(string.Format(@"Dear {0},",
xmlDocument.SelectSingleNode(@"/d/Customers/Customer/@Name").Value));
    sb.AppendLine("");
    sb.AppendLine("This is to let you know that your following Orders have been
confirmed.");
    sb.AppendLine("");
    sb.AppendLine("Order Details are ;");
    sb.AppendLine("Order Number                 Order Type              Order Amount");
    XmlNodeList orderList = xmlDocument.SelectNodes(@"/d/Customers/Orders/Order");
    foreach (XmlNode node in orderList)
    {
        sb.AppendLine(string.Format(@"{0}                              {1}
{2}",
            node.Attributes["No"].Value, node.Attributes["Type"].Value,
node.Attributes["Amount"].Value));
    }
    sb.AppendLine("");
```

```
    sb.AppendLine("Thanks for doing business with us.");
    sb.AppendLine("");
    sb.AppendLine("Regards,");
    sb.AppendLine("Team");
    string outString = sb.ToString();
}
```

This approach indeed gives result which can easily be emailed to the respective user.

```
Dear Irfan,

This is to let you know that your following Orders have been confirmed.

Order Details are ;
Order Number                Order Type              Order Amount
1                           ABC                      1111
2                           DEC                      222

Thanks for doing business with us.

Regards,
Team
```

If you closely look at the above approach you will convince that the approach has a severe flaw, it is not maintainable at all. The approach do not give flexibility to add/change content of the message by the business user as message is hardcoded and require full development cycle. Hardcoded message in the code is severe Crime and should be punishable by paying bill of the party to whole development team.

Secondly what if business users require a new column **order status** to be included in the message. This would mean change in the text, change in the data and change in the code to understand new field hence a full development cycle is required.

An ideal approach would be to build a mechanism that gives flexibility to change the content as and when required by the Business User without going into full development cycle.

Solution

If analyze above approach we can find three ingredients that are making the recipe.

- The static Content that do not change for example "Thanks for doing business with us"
- Content data that may change For example Name of the Customer and Order Details
- Mechanism that link the content with the content data

The first step towards reaching a solution is to isolate the hardcoded message from the code. The message should be placed somewhere else maybe in the database (usually the case) or at file system.

The second step will naturally be to identify the content that gets changed over content that do not change or static in nature. As standard practice suggest we need to templatize the message to clearly

identify the difference between static content and dynamic content. For example in our case **Irfan** is the name of the customer that is dynamic in nature hence we can use **<CUSTOMER_NAME>** in the message. A templatize version of the message would be like

One can now see the benefit of using a template based message is that message can be separated easily with the code. Let's revisit our earlier code and modify it to use above template and see how much have we improved.

First Iteration

As elaborated below we have made a couple of changes in the above code. Message content is now moved in a separate file TextTemplate2.txt. Similarly the customer data has also been placed in a separate file Data2.xml. Both data and template are now being isolated.

C#

Shrink ▲

```
static void Attemp2()
{
    XmlDocument xmlDocument = new XmlDocument(), dom = new XmlDocument();

    string message = System.IO.File.ReadAllText(@"TextTemplate2.txt");
    string xmlData = System.IO.File.ReadAllText(@"Data2.xml");
    xmlDocument.LoadXml(xmlData);

    message = message.Replace(@"<CUSTOMER/>",
xmlDocument.SelectSingleNode(@"/d/Customers/Customer/@Name").Value);
    Regex regEx = new Regex(string.Format(@"<{0}>(.*\s)*</{0}>", "ORDERS"),
RegexOptions.IgnoreCase | RegexOptions.Multiline);
    Match match = regEx.Match(message);
    string templateString = match.Value;

    string holdString = string.Empty;

    XmlNodeList orderList = xmlDocument.SelectNodes(@"/d/Customers/Customer/Orders/Order");

    foreach (XmlNode node in orderList)
    {
        holdString += templateString;
        holdString = holdString.Replace(@"<ORDERS>", string.Empty);
        holdString = holdString.Replace(@"<ORDER_NUMBER/>", node.Attributes["No"].Value);
        holdString = holdString.Replace(@"<ORDER_TYPE/>", node.Attributes["Type"].Value);
        holdString = holdString.Replace(@"<ORDER_AMOUNT/>",
node.Attributes["Amount"].Value);
        holdString = holdString.Replace(@"</ORDERS>", string.Empty);
    }

    message = regEx.Replace(message, holdString);
}
```

The immediate benefit we get as expected is that Business User now at least can change the static content easily over production environment without any help. No development is required for a change in the text. Apart from that code has now been much cleaner and only addresses template code and its replacement logic. Adding a new column is now a bit easy, we only need to modify the Data.xml and adding a new line in the for each loop and that is it but still development cycle is required for all such instances.

The tricky part in the above code is the <ORDERS> code. ORDERS code is a repeatable code which would repeat the number of times an order present



```
<ORDERS>
    <ORDER_NUMBER/>                    <ORDER_TYPE/>                    <ORDER_AMOUNT/>
</ORDERS>
```

In our case there are two orders that make it repeated twice.

	ORDER_NUMBER	ORDER_TYPE	ORDER_AMOUNT

	ORDER_NUMBER	ORDER_TYPE	ORDER_AMOUNT
Order 1	1	ABC	1111
Order 2	2	DEC	2222

Regular Expression is a powerful tool that helps in searching and replacing content of a particular pattern. RegEx can be used in a similar way to identify a repeating code from start till end. For example in bellow message, content present inside the ORDERS tag can easily be identified using RegEx. A simple regex could be **<ORDERS>(.*\s)*</ORDERS>** which return static content between the ORDERS tag.

Once we get the template content, repeating it is easy as elaborated in the code. First iteration over

the code yields same text but in a more elegant way. But we are still far from our objective. The next thing we now need to look at is the mechanism that is linking the code with the data. In the Second Iteration we will generalize the data fetching mechanism.

Second Iteration

Again back to analysis stage, if we look at the way we are resolving template codes, we will notice that we are using XPATH for the template resolution. The beauty of XPATH is, it's very powerful and can cover complex situations. Truly speaking there is a reason of choosing XML as an underlying data source and the reason of course is XPATH. The most significant attribute of XPATH is that it can be isolated from the code. Yes, we can put XPATH in a separate file their by generalizing the template resolution mechanism. We can create a Meta configuration xml which will hold all the template codes along with their resolution functions in the form of XPATH at one place. A sample metaconfiguration.xml can be like

XML

```xml
<?xml version="1.0" encoding="utf-8" ?>
<ROOT>
  <CODE NAME="CUSTOMER_NAME"
        MULTIPLERESULT="FALSE"
        DATAFUNCTION="/d/Customers/Customer/@Name"/>
  <CODE NAME="ORDERS"
        MULTIPLERESULT="TRUE"
        DATAFUNCTION="/d/Customers/Customer/Orders/Order/@No"/>
  <CODE NAME="ORDER_NUMBER"
        MULTIPLERESULT="FALSE"
        DATAFUNCTION="/d/Customers/Customer/Orders/Order[@No='{0}']/@No"/>
  <CODE NAME="ORDER_AMOUNT"
        MULTIPLERESULT="FALSE"
        DATAFUNCTION="/d/Customers/Customer/Orders/Order[@No='{0}']/@Amount"/>
  <CODE NAME="ORDER_TYPE"
        MULTIPLERESULT="FALSE"
        DATAFUNCTION="/d/Customers/Customer/Orders/Order[@No='{0}']/@Type"/>
</ROOT>
```

The above XML is a Meta Configuration that acts as a database for all Template Codes along with their resolution functions. For example CUSTOMERS code will be resolved by applying the XPATH "/D/CUSTOMERS/CUSTOMER/@ID" over below XML data source.

XML

```xml
<d>
  <Customers>
    <Customer Name='Irfan'>
      <Orders Total='99999.99'>
        <Order No='1' Type='ABC' Amount='1111'/>
        <Order No='2' Type='DEC' Amount='222'/>
      </Orders>
    </Customer>
```

```
  </Customers>
</d>
```

All three ingredients, template code, customer data and the link between code and data has been separated from the code their by giving us complete control to manipulate the way we want. Let's modify our code one more time to incorporate this newly learned technique and see how it affects our code.

Just for illustrative purpose we can write the concept in the form of a pseudo code

VB

```
Begin
    Var  message = Read Template Message
    Var CustomerDataXml =  Read CustomerData Xml
    Var MetaConfiguratio = Read MetaConfiguration.xml
    Transform the message into a temporary xml string by following below rule <root> +
message + </root>
    Load Xml String in the XmlDocument
    Var XmlNodeList =  Get All Xml Nodes by applying XPath(@"/root/*")
    For each TemplateCode in XmlLNodeList
    Begin
        Var TempCodeXPath = MetaConfiguration[TemplateCode]
        Var TempCodeValue = ResolveXPath(TemplateCode, CustomerDataXml,
                                         MetaConfigruationXml,TempCodeInnerText);
        message = Replace TemplateCode in Message with TempCodeValue
    End
    Print Message
End
```

The concept is very simple and uses the power of XPath, let's materialize the concept by revisit our earlier code and see the affect.

C# Shrink ▲

```
static void Attemp2()
{
    XmlDocument xmlDocument = new XmlDocument(), dom = new XmlDocument();
    Dictionary<string, Tuple<string, string>> metaConfiguration = new Dictionary<string,
Tuple<string, string>>();

    XmlNodeList nodeList = null;

    string message = System.IO.File.ReadAllText(@"TextTemplate2.txt");
    string xmlData = System.IO.File.ReadAllText(@"Data2.xml");

    metaConfiguration = ReadMetaConfiguration();

    xmlDocument.LoadXml(xmlData);

    string xmlSourceMessage = string.Format("<{0}> + {1} + </{0}>", "root", message);

    dom.PreserveWhitespace = true;
    dom.LoadXml(xmlSourceMessage);
```

```
    nodeList = dom.SelectNodes(@"/root/*");

    foreach (XmlNode XNode in nodeList)
    {
        string resolvedTagValue = Resolve(XNode.Name, xmlDocument, metaConfiguration,
XNode.InnerXml);
        message = message.Replace(XNode.OuterXml, resolvedTagValue);
    }
}
```

Dictionary is an ideal data structure for quick lookup of Template Codes. Below code fragment read the xml file and transform data into Dictionary to eliminate template code lookup complexity.

C#

```
static Dictionary<string, Tuple<string, string>> ReadMetaConfiguration()
{
    Dictionary<string, Tuple<string, string>> dictionary = new Dictionary<string,
Tuple<string, string>>();

    XmlDocument metaXmlDocument = new XmlDocument();
    string metaData = System.IO.File.ReadAllText(@"MetaData.xml");
    metaXmlDocument.LoadXml(metaData);
    XmlNodeList nodeList = metaXmlDocument.SelectNodes(@"/ROOT/CODE");

    foreach (XmlNode xmlNode in nodeList)
    {
        string name = xmlNode.Attributes["NAME"].Value;
        string dataFunction = xmlNode.Attributes["DATAFUNCTION"].Value;
        string multiResult = xmlNode.Attributes["MULTIPLERESULT"].Value;
        dictionary[name] = new Tuple<string, string>(multiResult, dataFunction);
    }

    return dictionary;
}
```

The actual template code resolution logic is defined in Recursive manner. The function recursively identify a template code in the message, resolve the code using CustomerData xml and replace the template code with the value.

C# Shrink ▲

```
static string Resolve(string code, XmlDocument dataSource, Dictionary<string, Tuple<string,
string>> metaConfiguration, string dataContext)
{
    string outString = string.Empty;
    Tuple<string, string> codeAttrib = metaConfiguration[code];
    string xPath = string.Format(codeAttrib.Item2, dataContext);

    XmlNodeList nodeList1 = dataSource.SelectNodes(xPath);

    if (nodeList1.Count == 1)
    {
```

```
        outString = nodeList1[0].Value;
    }
    else if (nodeList1.Count > 1)
    {
        string multiResult = codeAttrib.Item1;
        string innerXmlMessage = string.Format("<{0}> + {1} + </{0}>", "root",
dataContext);
        XmlDocument innerDocument = new XmlDocument();
        innerDocument.LoadXml(innerXmlMessage);
        string origInnerMessage = dataContext;
        XmlNodeList innerNodeList = innerDocument.SelectNodes(@"/root/*");

        foreach (XmlNode node in nodeList1)
        {
            outString = outString + origInnerMessage;
            foreach (XmlNode XNode in innerNodeList)
            {
                string resolvedTagValue = Resolve(XNode.Name, dataSource, metaConfiguration,
node.Value);
                outString = outString.Replace(XNode.OuterXml, resolvedTagValue);
            }
        }
    }
    return outString;
}
```

The above code fragment will give us flexibility to change the message entirely on Production Environment. We can not only change static text of the message but now able to add new template code in the message. All is needed is a new template code in Meta Configuration Xml, the corresponding XPath for the template in Customer Data Xml and that is all is required.

For the sake of understanding we are not closing our Optimization cycle. There are still more complex cases that the above approach won't be able to address, Lets also make our message complex by switch to HTML as in real life text messages are almost obsolete.

HTML Shrink ▲

```
<html>
<body>

<h1 align='center'>Sample Report</h1>

<div id="accordion">

       <CUSTOMERS>
       <h3><CUSTOMER_NAME/></h3>
       <div>
              <table id="t01">
                <tr>
                  <th>Order Number</th>
                  <th>Order Amount</th>
                  <th>Order Type</th>
                </tr>
```

```
                <CUSTOMER_ORDERS>
                <tr>
                  <td><ORDER_NUMBER/></td>
                  <td><ORDER_AMOUNT/></td>
                  <td><ORDER_TYPE/></td>
                </tr>

                </CUSTOMER_ORDERS>
              </table>
       </div>
       </CUSTOMERS>
</div>
</body>
</html>
```

If you could closely look at above message you will notice that template codes are two levels deep.

XML

```
<CUSTOMERS>
      <CUSTOMER_NAME>
          <CUSTOMER_ORDERS>
          <ORDER_NUMBER/>
          <ORDER_AMOUNT/>
          <ORDER_TYPE/>
      </CUSTOMER_ORDERS>
</CUSTOMERS>
```

Customers template code represent number of customers and for each customer their orders tag will be evaluated. Raw template code form will be like this

Customer	Order	<ORDER_NUMBER/>	<ORDER_AMOUNT/>	<ORDER_TYPE/>
Asif	1	1	1111	ABC
	2	2	2222	DEC
Bharat	3	3	3333	GHI
	4	4	4444	JKL

The complexity of the situation increases if start adding iterative tags, increasing the depth. This speaks for one more optimization iteration

Third Iteration – Build a Messaging Library

We have now reached to a point where a designated messaging library seems a good and obvious choice. It is painful to do repetitive effort over doing same thing on all projects as messages whether in the form of Emails or Reports are integral part of any system. Using an independent library will significantly reduce the development effort and also provide maintainability through version management.

Following standard practice this library will also exposes Interface.

The idea is to have a framework that act as a building block for standard business applications with biasness towards Banking Application. Dilizity is the materialization of this idea. Right now the library only covers templatization part of a message. Soon we will add more layers in the framework like Data Access, Communication, Presentation, Audit, Reporting, Logging, security, etc. Let's take a look at each individual component of messaging library briefly

- **IDataContext**: Because of the hierarchal nature of template codes we need to have a mechanism to build stack to pass context to the template codes which are context sensitive. For example in below case <CUSTOMER_NAME> is context sensitive hence we need to pass the context to the tag to identify which customer are we refereeing here.
  <CUSTOMERS>

<CUSTOMER_NAME/>
<CUSTOMERS>

- **IMessagingDataAgent**: Underlying customer data can be in any form, it can be in XML, tabular or any other form. An interface is needed here to hide the internal mechanics of how data is represented.
- **ITagAgent**: Each tag can have its own science to get resolve and same is the case of it's semantic. We have used xml elements as tag; future version can have other way of representing a tag. Therefore an interface is needed here to hide the internal mechanics of how tags are represented and resolved.
- **IMessagingAgent**: This is the core Interface and responsible for template code resolution. There could be many versions a message; message can be in the form of simple text, html, Word Document, RTF, SMS, report, etc. If required each version can have its own way of template resolution hence an Interface is needed to isolate different version of implementations

The code follow same principles elaborated in different iterations therefore we don't need to repeat that explanation one more time. However we do like to show the end result after processing html template used above; but we do like to show the Meta configuration, customer data and the template.

We have used following Meta configuration

With following Customer Data

XML

```xml
<D>
  <CUSTOMERS>
    <CUSTOMER ID="0001" NAME='SYED ASIF IQBAL'/>
    <ORDERS TOTAL='99999.99'>
      <ORDER ID='1' TYPE='MOBILE' AMOUNT='2000.00'/>
      <ORDER ID='2' TYPE='LEDTV' AMOUNT='50000.00'/>
    </ORDERS>
    <CUSTOMER ID="0002" NAME='BHARAT GOSWAMI'/>
    <ORDERS TOTAL='99999.99'>
      <ORDER ID='3' TYPE='SHOE' AMOUNT='1500.00'/>
      <ORDER ID='4' TYPE='WATCH' AMOUNT='2000.00'/>
```

```
    </ORDERS>
  </CUSTOMERS>
</D>
```

And the End Result is

Points of Interest

Using the above stated rules the objective can also be achieved using a table in the database if one likes to avoid xml and XPath. Meta configuration is the key element that currently stored as XML, which can be moved to database table. Using database table as the source can have its own pros and cons. usually developer's find comfortability in using UPDATE SQL for modification purpose. In case of

a clustered environment change at the database is easy compared to file system changes at all servers in a cluster. Similarly in a corporate environment a single update over a database goes through a change management cycle and relatively hard to do if compared to file system changes.

Let's see what changes we foresee if we opt to use database as source of Meta configuration.

Configuration source in the table

The first step would be to create a table META_CONFIGURATION in the database. The table should contain a NAME, MULTIPLERESULT and DATA_FUNCTION fields. Source data can be populated in the form of INSERT SQL script.

The second step is to add a data access layer for accessing table in the database. The final step would be to read the table and process the XPath as we are processing earlier.

There are the minimum changed required to switch from an xml Meta configuration to a table base Meta configuration. But how about changing the whole XPath based mechanism to something more native to database. How about changing the XPath into SQL or to some scalar functions, Adopting to

SQL based strategy will make the library more robust. A though over this can be viewed as below. This will be the starting point of next major upgrade to the messaging library.

History

First Version

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Asif

Product Manager Avanza Solutions

Pakistan

Asif has started programming back in 1991 on 80286 8-16 MHZ systems. Starting from dBase III+, FoxPro, C, assembly (exceptional skills in Assembly language have added significant confidence in his development career). The programming saga continues exploring new technologies and languages ranging from C++, VC++, Java, Delphi, RPG400, SQL Server, Oracle to name a few and the exploration still continues to DOT Net Technologies, SOA architectures, BI, DSL, etc. These learning experiences are backed by strong theoretical background with a flavor of research.

Asif shows significant interest in reading fiction, biotechnology, Astronomy. He Loves watching movies and in his free time love to play with his kids.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

source code file link broken
Tridip Bhattacharjee 10-Aug-15 4:01

Re: source code file link broken
Asif 10-Aug-15 4:43

image are broken
Tridip Bhattacharjee 6-Aug-15 21:38

Re: image are broken
Asif 6-Aug-15 22:11

Re: image are broken
Nelek 7-Aug-15 5:30

Re: image are broken
Tridip Bhattacharjee 9-Aug-15 21:30

Re: image are broken
Nelek 10-Aug-15 9:17

Re: image are broken
Asif 10-Aug-15 23:54

Re: image are broken
Nelek 11-Aug-15 10:03

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2015 by _Asif_
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1